**ChatGPT disse:**

Para o projeto **Repositório Institucional + Assistente Virtual IA**, recomendo utilizar **WebSocket** para permitir comunicação em tempo real entre a interface web e o backend Python.

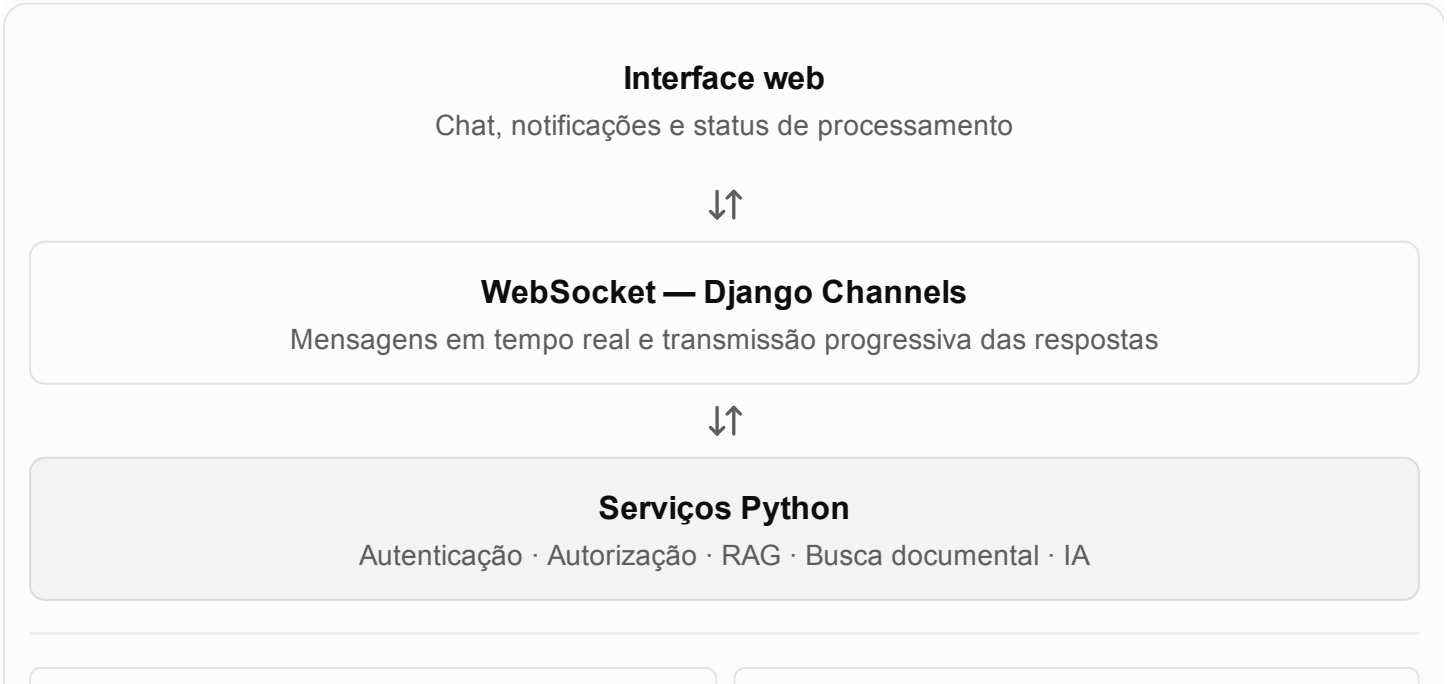
A escolha principal seria **Django + Django Channels + WebSocket**, mantendo o Django REST Framework para as operações tradicionais da aplicação.

## Socket.IO vs. WebSocket

| Característica | WebSocket nativo | Socket.IO |
|---|---|---|
| Comunicação bidirecional em tempo real | Sim | Sim |
| Reconexão automática | Deve ser implementada | Incluída |
| Salas e eventos | Implementação própria | Recursos integrados |
| Backend Python | Django Channels | Python Socket.IO |
| Cliente web | API WebSocket nativa | Cliente Socket.IO |
| Compatibilidade entre os protocolos | Protocolo padrão | Protocolo próprio sobre WebSocket ou outros transportes |

**Importante:** Socket.IO não é compatível diretamente com um servidor WebSocket puro. Cliente e servidor precisam usar protocolos correspondentes.

## Arquitetura recomendada



**PostgreSQL**
Dados e pgvector

**Repositório**
Documentos protegidos

**O que isso permitirá fazer?**

- Receber respostas da IA progressivamente, sem esperar a geração completa.
- Exibir o status “pesquisando documentos”, “consultando fontes” e “gerando resposta”.
- Atualizar notificações e o processamento de documentos em tempo real.
- Manter conversas simultâneas sem bloquear a interface.
- Preparar a aplicação para múltiplos usuários quando for instalada no servidor institucional.

**Segurança obrigatória**

O WebSocket deverá validar a identidade do usuário durante a conexão e autorizar cada conversa e inscrição em canais. Não basta autenticar somente no login: cada solicitação e cada evento devem respeitar as permissões vigentes por setor e documento. Mensagens e respostas não podem ser transmitidas a usuários não autorizados.

**Minha recomendação:** adotar `Django Channels + WebSocket + Django REST Framework` inicialmente. Essa combinação é adequada ao seu projeto Python e evita adicionar Socket.IO sem uma necessidade específica. Se o frontend já utilizar Socket.IO, podemos avaliar `python-socketio` como alternativa.